01

Intro to functions

Today

Welcome to your next lesson: Intro to functions.

I'm preparing your personalized lesson based on what you've learned, what's ahead, and the best ways to help it stick.

Step by step, you're moving forward.

Let’s connect this to what you already know 🔁

In Python you’d write:

Python
def add(a, b):
    return a + b
In JavaScript, the same idea with a slightly different outfit:

Javascript
function add(a, b) {
    return a + b;
}
Visually:

Python JavaScript
def function
:   { and }
indentation curly braces

Everything else (parameters, arguments, return) works the same idea-wise.

To start simple, I want you to write and run one function in the editor:

Open the code editor for this lesson.
Write this exact code and Run it:
Javascript

```
function greet() {
    return "Hello from JS!";
}

console.log(greet());
```

Once it runs, hit Submit code so I can see it.

No changes yet — just get this working, then I'll help you build your own.

Javascript

```
function greet() {
    return "Hello from JS!";
}

console.log(greet());
```

Nice, Ron — that's a clean first JS function ✅

You just did two things:

Declared a function with function greet() { ... }
Called it inside console.log(greet()); so the returned value shows up

So greet() → returns "Hello from JS!", and console.log(...) prints it.

Next step: let's make a function that takes an input.

In Python you might do:

Python

```
def shout(name):
    return name.upper()
```

In JavaScript we mirror that shape:

Javascript

```
function shout(name) {
    return name.toUpperCase();
}
```

I'd like you to:

In the editor, below your existing code, add:

```
89
90  function shout(name) {
91      return name.toUpperCase();
92  }
93
94  console.log(shout("Ron"));
95  Run it, then Submit code.
96
97
98  Don’t tell me the output yet — just get it running, then
    submit.
99
100 Javascript
101 function greet() {
102     return "Hello from JS!";
103 }
104
105 console.log(greet());
106 function shout(name) {
107     return name.toUpperCase();
108 }
109
110 console.log(shout("Ron"));
111 Nice work — that’s exactly right 💪
112
113
114 You’ve just:
115
116
117 Passed an input ("Ron") into shout
118 Used that input via the parameter name
119 Returned a new value with return name.toUpperCase();
120
121 So:
122
123
124 name (in the function definition) = parameter
125 "Ron" (when you call shout("Ron")) = argument
126 Let’s now focus on return vs console.log — they’re easy to
    mix up.
127
128
129 Quick idea:
130
131
132 return = gives a value back to whoever called the function
    (data flow)
```

```
console.log = just prints something on the screen (for humans)

Make sense so far, or feels fuzzy?

That makes sense

Great 👍


Let’s test that understanding with a tiny experiment.


Right now you have:

Javascript
function shout(name) {
    return name.toUpperCase();
}

console.log(shout("Ron"));
I want you to change it so that:


Inside shout, you use console.log(name.toUpperCase());
And you do not return anything from shout (remove the return)

So it should look like:

Javascript
function shout(name) {
    console.log(name.toUpperCase());
}

console.log(shout("Ron"));
Steps:


Update the code in the editor to match that.
Run it.
Then hit Submit code so I can see what happened.
it gets undefined error
Ron
undefined

function shout(name) {
console.log(name.toUpperCase());
}
```

```
console.log(shout("Ron"));
```

Perfect, you just hit an important JS behavior 👀

Let’s unpack what you saw:

Your code:

```
Javascript
function shout(name) {
    console.log(name.toUpperCase());
}

console.log(shout("Ron"));
```

This does two logs:

Inside the function: console.log(name.toUpperCase()); → prints RON

Outside: console.log(shout("Ron"));

But shout("Ron") doesn’t have a return, so its result is JavaScript’s default: undefined.

So the second console.log prints undefined.

So output order:

RON (from inside the function)

undefined (from logging the function’s return value, which is missing)

Quick check-in:

Does it now feel clear why you saw RON and then undefined? 🎯

Or should I draw a little “flow diagram” of the calls and outputs?

That makes sense now

Nice. You just nailed the return vs console.log difference 🎯

Next, let’s play with default parameters — a parameter that

gets a fallback value if you don’t pass an argument.

Example shape:

Javascript
function greetPerson(name = "friend") {
    return "Hello, " + name;
}
If you call greetPerson("Ron") → uses "Ron"
If you call greetPerson() → uses the default "friend"
Your turn in the editor:

Keep your existing code if you like; below it, add:
Javascript
function greetPerson(name = "friend") {
    return "Hello, " + name;
}

console.log(greetPerson("Ron"));
console.log(greetPerson());
Run it, then Submit code.

After that, we’ll connect this to undefined and missing arguments.

Javascript
function shout(name) {
    console.log(name.toUpperCase());
}

console.log(shout("Ron"));

function greetPerson(name = "friend") {
    return "Hello, " + name;
}

console.log(greetPerson("Ron"));
console.log(greetPerson());
Beautiful — that’s exactly how default parameters work 🎯

You just saw:

greetPerson("Ron") → uses your argument → "Hello, Ron"

```
greetPerson() → no argument → uses default "friend" → "Hello, friend"

So when you don’t set a default, a missing argument becomes undefined.

When you do set a default, that default is used instead.

Next, let’s practice returning multiple values from a function using an array.


In the editor, below your current code, add this new function:

Javascript
function getNameParts() {
    const first = "Ron";
    const last = "Hensley";
    return [first, last];
}

const parts = getNameParts();
console.log(parts);
Then:


Run it.
Hit Submit code.

We’ll improve it after I see that run.

Javascript
function shout(name) {
    console.log(name.toUpperCase());
}

console.log(shout("Ron"));

function greetPerson(name = "friend") {
    return "Hello, " + name;
}

console.log(greetPerson("Ron"));
console.log(greetPerson());

function getNameParts() {
    const first = "Ron";
```

```
    const last = "Hensley";
    return [first, last];
}

const parts = getNameParts();
console.log(parts);
Nice, Ron — that's a proper "multiple values via array" function ✅


Quick picture of what you have:

Javascript
function getNameParts() {
    const first = "Ron";
    const last = "Hensley";
    return [first, last];   // ← returns an array
}

const parts = getNameParts();  // ← parts is now ["Ron", "Hensley"]
console.log(parts);
So parts is an array with 2 items:


index 0 → first name
index 1 → last name
Next tiny step: let's use those pieces.


In the editor, below your current code, add:

Javascript
console.log(parts[0]);
console.log(parts[1]);
Then:


Run it.
Hit Submit code.

After that, I'll have you try the same idea using an object instead of an array.

Javascript
function shout(name) {
    console.log(name.toUpperCase());
```

```
}

console.log(shout("Ron"));

function greetPerson(name = "friend") {
    return "Hello, " + name;
}

console.log(greetPerson("Ron"));
console.log(greetPerson());

function getNameParts() {
    const first = "Ron";
    const last = "Hensley";
    return [first, last];
}

const parts = getNameParts();
console.log(parts);

console.log(parts[0]);
console.log(parts[1]);
Awesome — you just:


Defined a function that returns multiple values in an array
Stored the result in a variable
Pulled out each value with parts[0] and parts[1] 💪

That covers the “return multiple values with an array”
outcome.

Now let’s do the same with an object, which is often clearer
when values have names.


In the editor, below your current code, add:

Javascript
function getNameObject() {
    const first = "Ron";
    const last = "Hensley";
    return { first: first, last: last };
}

const nameObj = getNameObject();
console.log(nameObj);
```

```
console.log(nameObj.first);
console.log(nameObj.last);
Then:


Run it.
Hit Submit code.

We’ll check that, and then I’ll do a quick mastery check for the whole lesson.

Javascript
function shout(name) {
    console.log(name.toUpperCase());
}

console.log(shout("Ron"));

function greetPerson(name = "friend") {
    return "Hello, " + name;
}

console.log(greetPerson("Ron"));
console.log(greetPerson());

function getNameParts() {
    const first = "Ron";
    const last = "Hensley";
    return [first, last];
}

const parts = getNameParts();
console.log(parts);

console.log(parts[0]);
console.log(parts[1]);

function getNameObject() {
    const first = "Ron";
    const last = "Hensley";
    return { first: first, last: last };
}

const nameObj = getNameObject();
console.log(nameObj);
console.log(nameObj.first);
console.log(nameObj.last);
```

This is excellent, Ron 🎯

You've now shown all the key pieces:

Define functions with function and use return
Pass inputs via parameters (like name) and call them with arguments
Return multiple values using:
an array: return [first, last];
an object: return { first: first, last: last };

Quick visual of your last function:

Javascript
```
function getNameObject() {
    const first = "Ron";
    const last = "Hensley";
    return { first: first, last: last };  // bundle into one object
}

const nameObj = getNameObject();          // { first: "Ron", last: "Hensley" }
console.log(nameObj.first);               // "Ron"
console.log(nameObj.last);                // "Hensley"
```
From your code, you've hit all the mastery outcomes for this lesson ✅

Are you ready to wrap up this Intro to functions lesson and move on to the next one, or do you want to practice one more custom function (maybe with your own data)?

Love it 🚀

You wrapped up:

Translating Python-style functions to JavaScript function syntax
Using parameters and arguments cleanly

```
Understanding return vs console.log
Returning multiple values via arrays and objects

This sets you up really well for the next functions lessons, where functions start to power more of your JS logic.



Before we jump ahead:

Do you feel more curious about anything specific in functions (returns, defaults, arrays vs objects) that you’d like a 1–2 sentence recap on?


function greet() {
    return "Hello from JS!";
}

console.log(greet());
function shout(name) {
    return name.toUpperCase();
}

console.log(shout("Ron"));

function shout(name) {
    console.log(name.toUpperCase());
}

console.log(shout("Ron"));

function greetPerson(name = "friend") {
    return "Hello, " + name;
}

console.log(greetPerson("Ron"));
console.log(greetPerson());

function getNameParts() {
    const first = "Ron";
    const last = "Hensley";
    return [first, last];
}

const parts = getNameParts();
console.log(parts);
```

```
531
532   console.log(parts[0]);
533   console.log(parts[1]);
534
535   function getNameObject() {
536       const first = "Ron";
537       const last = "Hensley";
538       return { first: first, last: last };
539   }
540
541   const nameObj = getNameObject();
542   console.log(nameObj);
543   console.log(nameObj.first);
544   console.log(nameObj.last);
545
```